Detailed Author Response

Dear Area Chair and Reviewers,

We sincerely thank the Area Chair and Reviewers for their time and effort in evaluating our manuscript. We appreciate their recognition of the importance of Text-to-RLS, the value of RLSbench, and the effectiveness of the scope–anchor design in CARGO. We have carefully considered every comment. Below, we provide point-by-point responses, report only experimental evidence that directly addresses the reviews, and separate measured evidence from conditional camera-ready clarification.

Every end-to-end result table reports the same six metrics: policy-creation success (PCS), micro behavioral accuracy over executable tests (BA), item-level over-permission rate (OPR), item-level under-permission rate (UPR), missing-context default-deny pass rate (DDeny), and strict item accuracy (IA). IA requires successful policy creation and passage of every test for an item. OPR and UPR are lower-is-better; the other four metrics are higher-is-better. Anchor exact match in the extractor ablation is a separate diagnostic.

Response to Reviewer #1

C1: Scope of the contribution and access-control claims

> **Reviewer comment.** *The scope is narrower than the paper's broader access-control framing suggests. The contribution is specifically about PostgreSQL row-level security policy synthesis, not general access-control policy synthesis. So the authors should have indicated this directly in the paper title. RLS can express useful combinations of role, subject attributes, resource attributes, and runtime variables. However, enforcement remains a Boolean row predicate attached to a table and command. The method therefore does not cover many access-control mechanisms needed in modern data systems, including: (i) column-level and cell-level authorization; (ii) dynamic data masking and redaction; (iii) result-dependent controls, such as limiting aggregates or preventing inference; (iv) query-shape restrictions; (v) purpose limitation and consent; (vi) obligations and post-access actions; (vii) usage control after data release; cross-service authorization; (viii) authorization over files, streams, vectors, graphs, or unstructured objects; (ix) policies requiring external policy decision points. The authors should change claims such as "reliable generation of database access-control policies" to "PostgreSQL RLS policies" unless they provide evidence of broader applicability.*

Response to C1.

We thank the reviewer for pointing out that the paper's broader access-control framing exceeds the scope actually evaluated. We agree and will revise the paper accordingly. Our evaluated contribution is specifically the synthesis of PostgreSQL Row-Level Security (RLS) policies from natural-language authorization requirements. Within this scope, Text-to-RLS supports authorization requirements that can be expressed as row-level predicates over protected relations and operations, including combinations of roles, runtime attributes, row attributes, relational conditions, and environmental constraints.

Accordingly, we will change the title to *Text-to-RLS: Constraint-Anchored Synthesis of PostgreSQL Row-Level Security Policies from Natural Language* and narrow claims such as database access-control policy generation" to PostgreSQL RLS policy generation" throughout the paper. We will also explicitly state that Text-to-RLS does not cover column- or cell-level authorization, dynamic masking or redaction, result- or inference-dependent controls, query-shape restrictions, post-access obligations, usage control after data release, cross-service authorization, non-relational objects, or authorization requiring external policy decision points.

At the same time, the scope of the current evaluation should be distinguished from the potential portability of the approach. CARGO represents an authorization requirement through a protected relation and operation, together with runtime, row, and environment constraints. These abstractions describe predicate-based row authorization rather than PostgreSQL syntax itself. PostgreSQL-specific constructs such as `CREATE POLICY`, `USING`, `WITH CHECK`, policy composition, and execution behavior are introduced when these constraints are rendered, validated, and evaluated. Therefore, extending

Text-to-RLS to relational DBMSs with comparable predicate-based row-security mechanisms would mainly require adapting these DBMS-specific components and their execution semantics. We will make this distinction explicit while making no empirical claim of cross-DBMS generalization, since the present work evaluates PostgreSQL only.

C2: Structured row attributes versus semantic content

> **Reviewer comment.** *Content-based access control., which is critical for databases, is included only in a restricted structured-data sense. The approach does inspect row values such as status, amount, severity, tenant, or clearance. In that sense, it supports resource-attribute or structured-content-based policies. However, it does not address policies based on the semantic content of data. Examples include: (i) allow access only to documents that do not contain personal health information; (ii) permit analysts to view messages unrelated to an active investigation; (iii) redact records containing trade secrets; (iv) restrict images based on recognized persons or locations; (v) classify documents dynamically before release. Such policies require classifiers, content inspection, labels, provenance, or external functions, rather than ordinary RLS predicates over pre-existing columns.The authors do not explicitly distinguish: (i) authorization based on structured row attributes; and (ii) semantic content-based access control. The broader terminology used in the paper could lead readers to believe that the latter is supported.*

Response to C2.

We thank the reviewer for highlighting this important distinction. We agree that the paper did not clearly distinguish between structured row-attribute authorization and semantic content-based access control. CARGO supports authorization predicates over structured attributes already represented in the relational schema or trusted runtime context, such as `tenant_id`, `status`, `severity`, `clearance`, and temporal conditions. This corresponds to structured attribute-based authorization [1]. In contrast, CARGO does not interpret the semantic content of unstructured data, such as detecting protected health information in free text, identifying trade secrets in documents, recognizing persons or locations in images, or determining whether a message is semantically related to an investigation.

Semantic content-based controls require an additional mechanism that derives content-dependent authorization predicates or trusted labels [2]. An RLS policy can consume a trusted label produced by such a mechanism, but deriving or validating that label falls outside the scope of the Text-to-RLS task. We will make this limitation explicit, distinguish structured row-attribute authorization from semantic content-based access control throughout the paper, and remove wording that could imply that Text-to-RLS performs semantic content inspection.

C3: Multi-policy interaction is not evaluated

> **Reviewer comment.** *Multi-policy interaction is central to RLS but is not evaluated. This is perhaps the most serious scope limitation of the work and my major concern. The authors correctly emphasize that PostgreSQL policies compose through permissive and restrictive semantics and that multiple policies may interact. RLSbench contains 60 multi-policy bundles. However, all reported CARGO experiments use only the 490 single-policy items; the multi-policy items are reserved for future work. This creates a mismatch between the motivation and the evaluated contribution. Many of the hardest RLS errors arise from interactions among: (i) multiple roles; (ii) permissive policies combined by OR; (iii) restrictive policies combined by AND; separate read and write policies; (iv) exceptions and deny-like constraints; (v) policies inherited through roles. The authors should have narrowed the task definition and claims to single-policy synthesis.*

Response to C3.

We evaluated all 60 released multi-policy bundles, covering 686 executable tests. Because the submitted CARGO pipeline synthesizes one policy at a time, an LLM first decomposes each bundled requirement into individual policy requirements. Each component is then processed independently by the original CARGO pipeline for scope–anchor extraction, demonstration retrieval, and policy generation. The generated policies are then installed together and evaluated jointly as the original bundle. Decomposition

errors remain part of the end-to-end score. A bundle receives IA credit only when the complete set of generated policies installs successfully and passes all associated authorized, unauthorized, missing-session, command-isolation, and write-boundary tests.

For the direct baselines, Gemini-3.1-Pro generates policies directly from the original bundled requirement without the CARGO decomposition/extraction pipeline. The 0-shot baseline receives no demonstrations, while the 5-shot baseline uses five fixed-seed random enterprise demonstrations rather than retrieved examples. CARGO uses the pipeline with DeepSeek-V3.2 as the frozen policy generator. To separate the pipeline's contribution from that of the generator backbone, we also evaluate Gemini-3.1-Pro using the same pipeline.

Table 1. Multi-policy results across 60 bundles.

System	PCS↑	BA↑	OPR↓	UPR↓	DDeny↑	IA↑
CARGO (DeepSeek-V3.2)	0.983	0.862	0.109	0.318	0.967	0.483
CARGO (Gemini-3.1-Pro)	1.000	0.828	0.138	0.417	0.987	0.367
Gemini-3.1-Pro, 0-shot	1.000	0.668	0.258	0.854	1.000	0.067
Gemini-3.1-Pro, 5-shot	1.000	0.764	0.180	0.616	1.000	0.200

CARGO passes 29/60 complete bundles, compared with 12/60 for the best direct baseline and 22/60 for the same-pipeline Gemini control. Relative to direct Gemini-3.1-Pro 5-shot generation, CARGO improves bundle IA from 0.200 to 0.483 and reduces OPR from 0.180 to 0.109. Importantly, Gemini-3.1-Pro itself improves from 0.200 IA under direct 5-shot generation to 0.367 under the same pipeline. Under the matched pipeline, DeepSeek-V3.2 further improves IA from 0.367 to 0.483. The results show that the multi-policy gain is not solely due to the generator backbone. The CARGO pipeline provides a substantial improvement under Gemini as well, while DeepSeek-V3.2 contributes an additional gain. Across the full automatic evaluation, CARGO achieves 0.591 IA, combining 291 correct single-policy items and 29 correct multi-policy bundles, compared with 0.246 for the strongest baseline.

C4: Current accuracy is not sufficient for unattended deployment

> **Reviewer comment.** *The remaining error rate is too high for direct deployment. CARGO's improvement is substantial, but an item accuracy of 60.5% means that approximately 39.5% of generated policies fail at least one test. Even more concerning is that the reported over-privilege rate is 16%, meaning that a nontrivial fraction of executable generated policies allow at least one unauthorized case.*

Response to C4.

CARGO is designed as an assistive policy-authoring system. Generated policies should be treated as unverified drafts until they pass execution-based testing and administrator review, and failed or unverified outputs should be handled conservatively.

At the same time, we believe the current accuracy also reflects the difficulty of the newly formulated Text-to-RLS task. The strongest evaluated direct-generation baseline achieves only 0.252 IA, whereas CARGO reaches 0.605 and reduces OPR to 0.160. The generated policy can be syntactically valid yet behaviorally incorrect due to a missing constraint, an incorrect runtime attribute, an erroneous relational condition, or an asymmetric `UPDATE` predicate. At the same time, we believe the current accuracy also reflects the difficulty of the newly formulated Text-to-RLS task. The strongest evaluated direct-generation baseline achieves only 0.252 IA, whereas CARGO reaches 0.605 and reduces OPR to 0.160. The experimental results show that the generated policy can be syntactically valid yet behaviorally incorrect due to a missing constraint, an incorrect runtime attribute, an erroneous relational condition, or an asymmetric `UPDATE` predicate. These results suggest that Text-to-RLS remains a challenging and underexplored problem, with substantial room for further research in large-scale database settings involving complex and interacting authorization constraints

A similar progression occurred in Text-to-SQL. The original Spider benchmark reported only 9.7%

exact-match accuracy for its strongest model on the database split [11]. We do not equate SQL query accuracy with security policy safety, and authorization policies necessarily require stricter deployment standards. Rather, the Spider result illustrates that low initial accuracy on a newly established task does not reduce the value of defining the problem, constructing an executable benchmark, and developing methods that make systematic improvement measurable.

We therefore view the main contribution of this work as establishing Text-to-RLS as a systematic research problem and providing the task formulation, an executable benchmark, an evaluation methodology, and a constraint-anchored generation pipeline needed to study it. CARGO substantially improves over direct LLM generation, but the task remains unsolved, and further advances in generation and verification are required before autonomous deployment can be considered. The revision will make both points explicit: the current system requires verification and human review, while RLSBENCH and CARGO provide a foundation for measuring and improving future Text-to-RLS systems.

C5: Finite execution tests do not prove semantic equivalence

> **Reviewer comment.** *Execution tests do not establish semantic equivalence. The paper defines semantic correctness as agreement for every row, operation, and session state. However, evaluation uses a finite set of 3,546 test cases. Passing these tests does not establish equivalence for all database states and contexts. The authors should have avoided conflating test-suite equivalence with semantic equivalence. Stronger approaches might include: (i) symbolic execution; (ii) SMT-based equivalence checking;(iii) property-based test generation; (iv) mutation testing; iv) exhaustive evaluation over bounded domains.*

Response to C5.

We agree with the reviewer that finite execution tests do not prove semantic equivalence over all possible database states and runtime contexts. Equation (1) defines the ideal universally quantified correctness condition, whereas RLSBENCH measures behavioral agreement over a finite executable test suite. We will revise the terminology accordingly and avoid describing test-suite agreement as a proof of semantic correctness.

This distinction also appears in Text-to-SQL evaluation. Exact-match and execution-based metrics are widely used because they provide practical and scalable measures of generated-query correctness, although neither establishes full semantic equivalence. Test-suite accuracy was introduced specifically to provide a stronger practical approximation to semantic accuracy than execution on a single database instance [12]. More recent work continues to study functional-equivalence metrics beyond conventional string and execution comparison [10]. These metrics, therefore, provide useful correctness signals without constituting formal equivalence proofs.

RLSBENCH follows the same practical evaluation principle but adapts execution testing to authorization behavior. Its positive, negative, write-boundary, command-isolation, and missing-context cases test whether generated policies permit and deny the intended operations under multiple runtime contexts. This execution-based evaluation detects behaviorally important RLS errors that string or AST comparison alone cannot capture, including omitted predicates, incorrect runtime anchors, default-deny violations, and asymmetric `UPDATE` behavior. PCS, BA, OPR, UPR, DDeny, and IA should therefore be interpreted as empirical behavioral metrics over the supplied test cases rather than certificates of universal equivalence.

Furthermore, developing a stronger equivalence metric for RLS policies is itself a substantial research problem. Such a metric would need to reason about joins, correlated subqueries, NULL semantics, temporal predicates, session-dependent values, and policy composition across arbitrary database states and contexts. Prior work on improved functional-equivalence evaluation for Text-to-SQL similarly treats metric design as a dedicated research problem [10]. We therefore consider symbolic, SMT-based, property-based, or other stronger verification methods for RLS policies important directions for future work rather than components that can be reliably added to the present evaluation.

C6: Synthetic benchmark and cross-schema bias

Reviewer comment. *The benchmark is largely synthetic and based on one designed enterprise schema. Although the benchmark spans eight named domains, they are modules of a single constructed enterprise database with 45 tables. The requirements and reference policies are newly authored, and much of the supervision corpus is template-instantiated. This creates several possible biases: (i) common vocabulary and structural regularity; (ii) similar requirement-writing style; (iii) repeated policy templates; (iv) overlap between training templates and benchmark patterns; (v) schemas designed to make required attributes available; (vi) limited ambiguity compared with real requirements. The authors should have evaluated their approach on independently developed schemas and policies from actual applications. Cross-schema generalization is particularly important because schema grounding is a central claimed contribution.*

Response to C6.

We agree that RLSBENCH is a constructed benchmark and may inherit biases from model-assisted drafting, template-derived supervision, shared vocabulary, and authoring style. However, its automated and LLM-assisted construction steps are followed by substantial manual and executable validation. Each released requirement is grounded in a concrete schema and runtime-context specification, its reference policy is manually checked, and its intended authorization behavior is validated through executable positive, negative, and missing-context tests. These quality-control steps do not eliminate construction bias, but they prevent RLSBENCH from being an unchecked collection of LLM-generated policies.

RLSBENCH establishes a controlled pipeline that makes Text-to-RLS measurable and reproducible, including schema grounding, runtime constraints, write boundaries, default-deny behavior, and false-permit errors. Collecting independently authored requirements and executable policies from deployed applications would provide stronger ecological validity, but constructing such a corpus would require substantial application-specific annotation, policy validation, and development of executable tests. We consider this an important direction for future benchmark expansion.

To further test whether the observed gains depend on our designed enterprise schema, we additionally evaluate CARGO on OHDSI's independently maintained OMOP CDM v5.4 PostgreSQL schema [8]. The original schema contains 39 tables and 432 columns and is used without structural modification. We construct 40 healthcare authorization requirements that cover 15 Nested, 10 Combined, 5 Temporal, 4 Relational, 2 Recursive-Hierarchy, 2 Ordinal, and 2 Compliance patterns.

The runtime specification contains five healthcare-specific context variables covering provider/patient identity, care site, specialty, and purpose of use. The execution harness sets and resets these variables and includes tests for missing context. Importantly, CARGO's five in-context demonstrations are drawn exclusively from the enterprise source domain rather than from the healthcare evaluation requirements.

The experiment compares direct generation with the CARGO pipeline under two generator backbones. On the frozen 40-item holdout, GPT-5.4+CARGO improves IA from 0.45 to 0.75, increases BA from 0.638 to 0.953, and reduces OPR from 0.376 to 0.071.

These results provide preliminary evidence that the benefits of the CARGO pipeline extend beyond our enterprise schema and vocabulary. We will add this cross-schema evaluation to the camera-ready and identify the evaluation of deployment requirements for the application as future work.

C7: PostgreSQL specificity

Reviewer comment. *PostgreSQL specificity limits generalization (see also comment (1)). The method is tightly coupled to: PostgreSQL CREATE POLICY; USING and WITH CHECK; current_setting; PostgreSQL permissive/restrictive composition; PostgreSQL execution behavior. This is is fine for a focused systems paper, but the title and conclusions should make the scope explicit. Generalization to systems such as Oracle VPD, Oracle RAS, SQL Server security predicates, Snowflake row-access policies, BigQuery row policies, or middleware languages would require different syntax and semantics.*

Response to C7.

We agree that the current implementation and evaluation are PostgreSQL-specific. The generator,

validator, and execution harness instantiate PostgreSQL constructs and semantics, including `CREATE POLICY`, `USING/WITH CHECK`, `current_setting`, command-specific behavior, and policy composition.

At the same time, the core scope–anchor formulation represents an authorization requirement through a protected relation and operation, together with runtime, row, and environment constraints. These abstractions operate at the level of predicate-based row authorization. PostgreSQL-specific constructs are introduced only when the extracted constraints are rendered into executable policies and behaviorally validated.

Therefore, extending CARGO to another relational DBMS with comparable predicate-based row-security mechanisms would require adapting the backend-specific policy renderer, validator, and execution harness to that DBMS's syntax and semantics. We have not evaluated such an extension and therefore make no empirical claim of cross-DBMS generalization. We will revise the paper to clearly distinguish the portability of the core formulation from the PostgreSQL-specific implementation and evaluation.

Response to Reviewer #2

C1: Multi-policy bundles are excluded

> **Reviewer comment.** *The experiments exclude all multi-policy bundles, despite claims about policy bundles and permissive/restrictive composition.*

Response to C1.

We evaluated all 60 multi-policy bundles. Each bundled requirement is decomposed into individual policy requirements, processed by CARGO, and then installed and tested jointly as a complete policy set. Decomposition errors remain part of the end-to-end evaluation, and a bundle receives IA credit only when the complete set of generated outputs passes all associated tests. The evaluation contains 686 executable tests covering permissive-OR and restrictive-AND composition, command isolation, role interaction, write boundaries, and missing-session default deny.

As shown in Table 1, CARGO achieves 0.483 bundle IA, compared with 0.200 for the strongest direct 5-shot baseline. It also reduces OPR from 0.180 to 0.109 and UPR from 0.616 to 0.318. Thus, this evaluation directly measures interactions among jointly installed policies rather than evaluating their components independently.

Across the full automatic evaluation, CARGO achieves 0.591 IA, compared with 0.246 for the strongest baseline.

C2: Supabase is an exact adjacent product precedent

> **Reviewer comment.** *The paper appropriately draws practical RLS structures from Supabase guides and repositories. However, it does not analyze Supabase or other database platforms as adjacent software products, nor clarify how their existing policy-authoring facilities overlap with or differ from Text-to-RLS. Supabase is a direct market precedent: Supabase has offered an AI-assisted PostgreSQL RLS editor since 2023 and now supports plain-language RLS generation and RLS testing. This omission materially weakens the novelty discussion.*

Response to C2.

We thank the reviewer for pointing out Supabase as a direct software precedent. Supabase has supported AI-assisted RLS authoring since 2023; Assistant v2 generates policies from descriptions of desired behavior, and the recent RLS Tester supports query-based policy verification [5, 6, 7]. Compared to Supabase, our contribution is to formulate Text-to-RLS as a reproducible synthesis-and-evaluation problem and to provide an executable benchmark, behavioral evaluation protocol, and reproducible generation method.

We evaluate Supabase in two complementary settings. First, for a reproducible comparison, we use Supabase's publicly released RLS-generation prompt under the same generation and evaluation protocol

as our other prompted baselines. This prompted the Supabase baseline to achieve PCS 0.832, OPR 0.502, and IA 0.077 in the single-policy evaluation.

Second, we directly evaluate Supabase's *hosted project-level AI Assistant*. We deploy the RLSBENCH schema to a Supabase project and submit each of the 490 single-policy requirements in a separate conversation with *Schema Only* permission. Each conversation uses the same fixed zero-shot rule prompt and the unchanged natural-language requirement. We provide no target table, reference policy, demonstration, or sample-specific schema hint. The Assistant inspects the deployed schema through its native tools. We record the generated SQL without applying it to the hosted project, and retain missing, truncated, or invalid responses as failures.

Table 2. Single-policy comparison with Supabase.

System	PCS↑	BA↑	OPR↓	UPR↓	DDeny↑	IA↑
Supabase public prompt	0.832	0.557	0.502	0.717	0.697	0.077
Supabase hosted Assistant	0.840	0.626	0.418	0.722	0.779	0.100
CARGO	0.983	0.872	0.160	0.325	0.964	0.605

Within the 490-item evaluation subset, 444 hosted-Assistant responses contain an extracted `CREATE POLICY`. Under execution, 1,754/2,802 executed test cases pass, giving BA 0.626 and strict IA 0.100. Responses without an executable policy SQL still count as failures for PCS and IA.

Both Supabase evaluations remain substantially below CARGO under our evaluation protocol. The reproducible Supabase prompt obtains 0.077 IA and 0.502 OPR, while the hosted Assistant reaches 0.100 IA and 0.418 OPR. CARGO reaches 0.605 IA while reducing OPR to 0.160. These results show that natural-language RLS authoring clearly predates our work, but also that existing prompt- and product-level solutions leave substantial room for systematic Text-to-RLS methods and execution-based evaluation.

Because the hosted Assistant's complete model, prompting, and inference configuration are not public, the hosted experiment measures observable product behavior rather than reproducing its backend. The publicly released prompt provides the complementary reproducible baseline.

C3: Trusted runtime-context channel is unstated

> **Reviewer comment.** *The paper assumes a trusted runtime-context channel but does not state this assumption clearly. RLSbench injects identity and tenant attributes through PostgreSQL session settings, while the synthesis task treats these attributes as authoritative. The authors should clarify that authentication, secure propagation of session attributes, and prevention of caller-controlled spoofing are outside scope and state the trust boundary under which the generated policies are intended to be secure.*

Response to C3.

PostgreSQL exposes `current_setting` and `set_config` as interfaces for reading and changing session-local configuration, and accepts arbitrary two-part custom option names [3, 4]. We therefore treat these settings as a context-transport mechanism rather than an authenticated identity channel. The security claims in our work assume a trusted runtime-context channel with the following boundary:

1. A trusted authentication or middleware layer derives identity, role, tenant, department, and other authorization context from authenticated credentials.
2. Before executing an application transaction, the trusted layer binds the corresponding PostgreSQL session variables to that transaction. In deployments using pooled connections, these values must be transaction-local or reliably cleared and overwritten before the connection is reused.
3. Untrusted callers must not be able to control these runtime-context values or execute application queries through database roles that bypass the intended RLS boundary. In particular, superusers and roles with `BYPASSRLS` bypass RLS, and table owners normally do so unless `FORCE ROW LEVEL`

SECURITY is enabled [9]. Designing or verifying this trusted channel and database role configuration is outside the scope of the present synthesis task.

4. Missing runtime values are treated as fail-closed conditions by the benchmark policies and are explicitly exercised by the missing-session tests.

Under this threat model, CARGO synthesizes policies under the assumption that the runtime context Ω supplied to PostgreSQL is trustworthy. If an attacker can spoof values such as `app.tenant_id` or `app.employee_id`, even a behaviorally correct RLS predicate may enforce authorization against the attacker's false context. Authentication, secure claims propagation, connection-pool hygiene, database-role provisioning, and protection of runtime context integrity are therefore deployment responsibilities that fall outside the Text-to-RLS synthesis problem.

C4: Cross-database/schema generalization

> **Reviewer comment.** *The benchmark is synthetic and concentrated in one enterprise database; cross-schema generalization is not demonstrated.*

Response to C4.

We thank the reviewer for raising this concern. The submitted evaluation focuses on the designed enterprise schema, so it does not, by itself, establish that CARGO can operate beyond the schema and vocabulary used to construct RLSBENCH. We therefore added a cross-schema experiment on OHDSI's independently maintained OMOP CDM v5.4 PostgreSQL schema [8].

The upstream OMOP schema contains 39 tables and 432 columns and is used without structural modification. The requirements cover 15 Nested, 10 Combined, 5 Temporal, 4 Relational, 2 Recursive-Hierarchy, 2 Ordinal, and 2 Compliance patterns. The experiment changes both the database schema and application vocabulary from the enterprise setting to a healthcare domain.

For each requirement, the model receives the OMOP schema Σ_{OMOP} together with a healthcare-specific runtime specification Ω_{OMOP} containing `app.provider_id`, `app.patient_id`, `app.care_site_id`, `app.specialty_concept_id`, and `app.purpose_of_use`. The execution harness sets and resets these variables and includes missing-context cases. Importantly, CARGO's five in-context demonstrations are drawn exclusively from the enterprise source domain rather than from the healthcare evaluation set.

We compare direct 0-shot generation with the CARGO pipeline under two generator backbones, GPT-5.4 and DeepSeek-V3.2.

Table 3. Cross-schema performance on 40 OMOP requirements.

System	PCS↑	BA↑	OPR↓	UPR↓	DDeny↑	IA_{40} ↑
GPT-5.4 direct	0.700	0.638	0.376	0.326	0.655	0.450
GPT-5.4+CARGO	1.000	0.953	0.071	0.044	1.000	0.750
DeepSeek-V3.2 direct	0.975	0.832	0.208	0.152	0.910	0.350
DeepSeek-V3.2+CARGO	1.000	0.917	0.129	0.044	1.000	0.500

With GPT-5.4, CARGO improves IA from 0.450 to 0.750. On the full diagnostic set, BA increases from 0.638 to 0.953, OPR decreases from 0.376 to 0.071, and PCS increases from 0.700 to 1.000. The DeepSeek-V3.2 pairing shows the same direction: full-set IA increases from 0.350 to 0.500.

These results provide preliminary evidence that the benefit of the CARGO pipeline is not limited to the vocabulary and schema of the enterprise benchmark. Across both generator backbones, CARGO consistently improves behavioral correctness and item-level accuracy while reducing both over-permission and under-permission errors. This suggests that its constraint-anchored formulation and operation-aware generation remain effective under a substantially different healthcare schema and authorization vocabulary. We note, however, that this cross-schema evaluation is smaller in both schema scale and authorization-constraint coverage than the main RLSBENCH evaluation, which may partly explain the relatively strong

performance of the direct-generation baselines. Thus, rather than establishing broad cross-domain generality, this experiment provides initial evidence that CARGO's gains can transfer beyond the source schema and motivates evaluation on larger and more diverse schemas with broader constraint coverage.

C5: Artifact availability

> **Reviewer comment.** *The reviewer was unable to evaluate the code and benchmark because no artifact was available.*

Response to C5.

We apologize that the artifact was not reliably visible to every reviewer. Reviewer 3 could access the shared code and data, but Reviewer 2 could not. We have now verified from an unauthenticated session that the public repository is reachable at `https://github.com/Jae0v0/Text-to-RLS`.

The repository provides benchmark requirements, reference policies, schemas and runtime-context specifications, executable tests, code, predictions, evaluation scripts, and reproduction instructions.

Response to Reviewer #3

C1: Transfer to healthcare and need for per-domain fine-tuning

> **Reviewer comment.** *Limited demonstration that the proposed solution can be transferred to different domains with different style of database schemas.*

Response to C1.

We thank the reviewer for raising this important question. The CARGO interface is parameterized by a target schema Σ and a runtime-context specification Ω. Neither table/column names nor `app.*` runtime variables are hard-coded into the policy renderer. To examine domain transfer, we evaluated CARGO on OHDSI's independently maintained OMOP CDM v5.4 PostgreSQL schema, which contains 39 tables and 432 columns and uses a substantially different healthcare vocabulary from the enterprise benchmark. The evaluation contains 40 healthcare authorization requirements over 17 tables, and Ω_{OMOP} specifies five healthcare-specific runtime variables covering provider/patient identity, care site, specialty, and purpose of use. CARGO's five in-context demonstrations are drawn exclusively from the enterprise source domain.

CARGO receives only the target OMOP schema Σ_{OMOP} and the healthcare-specific runtime-context specification Ω_{OMOP}, while all five in-context demonstrations come exclusively from the enterprise source domain. Thus, the experiment directly tests whether the pipeline can transfer to a previously unseen schema and domain vocabulary without relying on target-domain examples.

The results show consistent gains across two generator backbones. With GPT-5.4, CARGO improves IA from 0.450 to 0.750, increases BA from 0.638 to 0.953 and PCS from 0.700 to 1.000, while reducing OPR from 0.376 to 0.071 and UPR from 0.326 to 0.044. With DeepSeek-V3.2, IA similarly improves from 0.350 to 0.500, with corresponding gains in behavioral correctness and permission safety.

These results provide direct preliminary evidence of cross-domain and cross-schema transfer: CARGO's benefits persist when both the database schema and authorization vocabulary change, without target-domain fine-tuning or healthcare demonstrations. We nevertheless avoid claiming universal cross-domain generalization, since the current evaluation has a smaller schema scale and constraint coverage than RLSBENCH. We will make this scope explicit and identify evaluation on larger and more diverse schemas as future work.

C2: Isolating decomposition from extractor fine-tuning

> **Reviewer comment.** *The evaluation does not fully isolate the benefit of fine-tuning from the benefit of decomposition.*

Response to C2.

We thank the reviewer for suggesting this controlled comparison. We ran the requested experiment by varying only the extractor while holding the remainder of the CARGO pipeline fixed. The prompted and fine-tuned extractors receive the same requirement, schema slice, runtime-context specification, label definitions, and structured output schema. Validation, operation-routed retrieval, five demonstrations, top-k, prompt budget, frozen DeepSeek-V3.2 generator, decoding configuration, database partition, and evaluator are identical across all extractor conditions.

Table 4. Controlled comparison of scope–anchor extractors.

Extractor	Anchor EM	PCS↑	BA↑	OPR↓	UPR↓	DDeny↑	IA↑
No extractor / direct	N/A	0.973	0.788	0.276	0.440	0.915	0.435
Prompted GPT-4.1	0.017	0.973	0.791	0.257	0.460	0.913	0.366
Prompted GPT-5.4	0.040	0.983	0.783	0.240	0.488	0.894	0.380
Fine-tuned f_θ	0.763	0.983	0.870	0.167	0.325	0.966	0.607
Gold/oracle anchors	1.000	0.996	0.911	0.113	0.265	0.991	0.680

The matched-extractor comparison shows that wrong decomposition alone does not account for CARGO's gain. With the rest of the pipeline fixed, GPT-4.1 and GPT-5.4 were prompted to achieve Anchor EM scores of only 0.017 and 0.040, respectively, and reach IA values of 0.366 and 0.380. Relative to direct generation, these prompted-extractor variants reduce OPR from 0.276 to 0.257 and 0.240, but do not improve strict IA. Thus, introducing the decomposition pipeline with inaccurately prompted anchors can reduce false permits while still degrading complete policy correctness.

The effect of extractor quality becomes clear when only the extractor is replaced. Replacing prompted GPT-5.4 with the fine-tuned f_θ raises Anchor EM from 0.040 to 0.763, increases IA from 0.380 to 0.607, and reduces OPR from 0.240 to 0.167. Relative to prompted GPT-4.1, IA increases from 0.366 to 0.607 and OPR decreases from 0.257 to 0.167. Because validation, retrieval, demonstrations, generator, decoding, and evaluation remain fixed in these comparisons, the improvement can be attributed to more accurate scope–anchor extraction rather than to the downstream CARGO components.

The gold-anchor control further supports this interpretation. Replacing f_θ with oracle anchors increases IA from 0.607 to 0.680 and reduces OPR from 0.167 to 0.113, indicating additional headroom from improving extraction accuracy. Overall, the experiment shows that the benefit of the decomposed CARGO pipeline depends on accurate scope–anchor extraction; decomposition alone is insufficient.

C3: Cross-domain transfer and need for per-domain fine-tuning

> **Reviewer comment.** *The benchmark is designed for a single database schema in a typical enterprise context, which is also similar to those corpus used for training/fine-tuning models in CARGO. One question is how transferable CARGO is if it is used for a different domain, e.g,. healthcare, where some special schema/contexts will be involved. Does it need to re-fine-tune CARGO for each different domain?*

Response to C3.

We thank the reviewer for raising this important question. CARGO is not tied to a fixed enterprise vocabulary at the interface level. It takes the target database schema Σ and runtime-context specification Ω as inputs rather than hard-coding table names, column names, or application-specific runtime variables. Therefore, adapting CARGO to a new domain does not inherently require retraining the entire pipeline. A new deployment can provide its own Σ and Ω while keeping the downstream retrieval, generation, validation, and execution components unchanged.

The extractor can also be instantiated without task-specific fine-tuning. In our controlled experiment, we replace the fine-tuned extractor with a prompted GPT-5.4 extractor while keeping validation, retrieval, demonstrations, the DeepSeek-V3.2 generator, decoding, and evaluation fixed. Without extractor fine-tuning, the complete pipeline still achieves PCS 0.983, BA 0.783, OPR 0.240, and IA 0.380. Thus,

a strong prompted model can directly instantiate the scope–anchor interface and generate executable policies without training a new extractor.

Fine-tuning nevertheless provides a substantial accuracy gain. Under the same fixed downstream pipeline, replacing the prompted GPT-5.4 extractor with the fine-tuned f_θ increases Anchor EM from 0.040 to 0.763, raises BA from 0.783 to 0.870, and IA from 0.380 to 0.607, and reduces OPR from 0.240 to 0.167. This comparison indicates that task-specific adaptation improves the identification of scope and authorization anchors, but it is not a prerequisite for operating the CARGO pipeline.

Our current evidence, therefore, suggests a practical trade-off. A prompted extractor can be used when no domain-specific training data are available, while fine-tuning is preferable when sufficient labeled requirements are available, and higher policy accuracy is required.

C4: Pipeline benefit versus extractor fine-tuning

> **Reviewer comment.** *The paper compares strong LLMs with the fine-tuned extractor on scope and anchor extraction, and separately compares direct-generation baselines. However, it does not run a strong prompted LLM as the extractor inside the complete CARGO pipeline while keeping validation, retrieval, and final generation fixed. Therefore, it remains unclear whether the gains mainly come from the scope-anchor pipeline or from the specialized fine-tuning.*

Response to C4.

We thank the reviewer for identifying this important distinction. We agree that the submitted evaluation does not include the exact control suggested by the reviewer: replacing the fine-tuned extractor with a strong prompted LLM extractor while keeping the complete downstream CARGO pipeline fixed. We therefore do not claim that the current experiments fully isolate decomposition from extractor fine-tuning.

However, the existing component ablation already shows that CARGO's end-to-end improvement cannot be attributed solely to the specialized extractor. In Table 5, the random 5-shot baseline achieves IA 0.193 and OPR 0.368. Adding the constraint-guided retrieval component G, without the fine-tuned extractor X, increases IA to 0.435 and reduces OPR to 0.276. Thus, a substantial part of the gain appears before introducing the fine-tuned scope–anchor extractor.

Table 5. Ablation of CARGO components on RLSBENCH.

Setting	X	G	V	OPR↓	IA↑
Baseline	×	×	×	0.368	0.193
+G	×	✓	×	0.276	0.435
+X+G	✓	✓	×	0.163	0.588
CARGO	✓	✓	✓	0.160	0.605

The ablation therefore separates two sources of improvement. First, the non-fine-tuned pipeline component G contributes a 24.2-point IA gain over the random 5-shot baseline and reduces OPR by 9.2 points. Second, adding the fine-tuned extractor X on top of the same retrieval component provides a further 15.3-point IA gain and reduces OPR from 0.276 to 0.163. The schema-validated reranker V then provides the remaining improvement to the full CARGO result.

These results show that specialized fine-tuning is important, but it does not account for the entire end-to-end improvement. Constraint-guided retrieval already provides a substantial gain without X, while accurate scope–anchor extraction provides an additional complementary benefit. We agree, however, that this ablation does not isolate the effect of the scope–anchor representation itself from the training procedure used to obtain X. A prompted LLM extractor inserted into the otherwise fixed full pipeline would provide a cleaner control for separating decomposition from fine-tuning, and we will identify this as an important additional ablation.

C5: Difficulty examples and concrete CARGO failures

Reviewer comment. *The paper should give examples for each difficulty category and analyze concrete cases where CARGO fails, rather than showing only baseline failures.*

Response to C5.

We map each benchmark-assigned difficulty label to a verified example and report its empirical failure rate. The increase from 26.2% to 53.4% provides evidence that the labels track structural difficulty rather than only surface length.

Table 6. CARGO failure rates by benchmark difficulty.

Level	Items	Failures	Failure rate
Simple	107	28	26.2%
Moderate	159	63	39.6%
Hard	157	68	43.3%
Extra-Hard	58	31	53.4%

Table 7 follows the reviewer's suggested format. It gives the requirement, relevant schema slice, reference-policy behavior, and structural reason for each label. The schema row includes only policy-relevant columns from the full 45-table schema.

Table 7. Examples at each benchmark difficulty level.

Field	Case description
Case	Simple, Item 129
Requirement	Compliance officers can access documents with document type Patent Application.
Schema slice	`operations.documents(document_id, document_type)`; runtime context `app.employee_id`, `app.job_role`.
Reference policy	`FOR SELECT USING` requires non-null employee/role context, `job_role = 'compliance_officer'`, and `document_type = 'Patent Application'`.
Why this level	One target table, one operation, one role equality, and one direct row predicate; no temporal condition, write boundary, Boolean branch, or subquery.
Case	Moderate, Item 342
Requirement	HR specialists can publish award nomination issues.
Schema slice	`engineering.issues(issue_id, issue_type, status)`; runtime context `app.employee_id`, `app.job_role`.
Reference policy	`FOR INSERT WITH CHECK` requires `job_role = 'hr_specialist'`, `issue_type = 'Award Nomination'`, and the implied post-state `status = 'Published'`.
Why this level	The policy is enforced on the write/new-row channel, and "publish" must be grounded to a concrete state even though the requirement does not spell out the column value.
Case	Hard, Item 72
Requirement	Customers can delete only their own draft orders in the current tenant during weekday business hours (09:00–18:00), before fulfillment.
Schema slice	`sales.orders(order_id, customer_id, tenant_id, status, fulfillment_status)`; runtime context `app.customer_id`, `app.tenant_id`, and trusted `app_now()`.
Reference policy	`FOR DELETE USING` conjunctively enforces identity, tenancy, `status = 'Draft'`, `fulfillment_status IS DISTINCT FROM 'Fulfilled'`, the 09:00–18:00 interval, and weekday `DOW` 1–5.
Why this level	Four independent constraint families must all survive: ownership, tenant isolation, two lifecycle predicates, and two temporal bounds. Omitting any conjunct changes authorization.
Case	Extra-Hard, Item 462
Requirement	Finance managers can read own-tenant invoices; cross-tenant invoices require an approved, unexpired, read-bearing financial-review grant for the current application user and a named approver.

Continued on next page

Table 7 continued

Field	Case description
Schema slice	`sales.invoices(invoice_id, tenant_id)` and `auth.access_grants(user_id, target_tenant_id, target_schema, target_table, can_read, financial_review, approved, approved_by, expires_at)`; runtime context `app.user_id`, `app.tenant_id`, `app.job_role`.
Reference policy	`FOR SELECT USING` requires the finance-manager role and then permits either the own-tenant branch or a correlated `EXISTS` branch whose grant is table-scoped, approved, unexpired, read-bearing, financial-review-specific, and names an approver.
Why this level	A top-level disjunction combines ordinary tenant access with a ten-attribute cross-table authorization path and a temporal validity check.

We then classify all 190 failures from the canonical CARGO run using a mutually exclusive fixed precedence. Table 8 gives one concrete item for every non-empty category, as requested; the category counts sum exactly to 190. FD and FP denote false deny and false permit, respectively; "both" denotes an executable policy exhibiting errors in both directions.

Table 8. Failure taxonomy for the canonical CARGO run.

Primary category	Count	Direction counts	Representative case and primary defect
Schema-invalid / non-executable	8	N/A	Item 436: generated predicate does not match the evaluator's risk-level column, so all 10 tests error.
Wrong table or command scope	9	1 FD, 0 FP, 8 both	Item 256: targets `operations.security_events` instead of `operations.audit_logs`.
Mis-grounded runtime anchor	10	3 FD, 1 FP, 6 both	Item 99: substitutes a table lookup for `app.position_level` and counts the wrong event table.
Wrong `USING`/`WITH CHECK` channel	0	—	No representative case: operation-channel routing makes zero errors in this run.
Nested / recursive structure	2	0 FD, 0 FP, 2 both	Item 92: removes the employee–tenant `EXISTS` path and changes `created_at` to `updated_at`.
Temporal / NULL handling	18	13 FD, 3 FP, 2 both	Item 94: drops the joined employee's non-null hire-date guard and timestamp cast.
Dropped constraint	48	15 FD, 13 FP, 20 both	Item 242: grounds assignment to `assignee_id` rather than `oncall_assignment`, and severity to priority.
Over-constraint / extra conjunct	15	8 FD, 1 FP, 6 both	Item 445: adds an unrequested self-assignment condition to a tenant-scoped insert policy.
Wrong literal value	55	24 FD, 2 FP, 29 both	Item 33: predicts case-sensitive `'Draft'` where the reference requires `'draft'`.
Predicate logic / join error	10	4 FD, 6 FP, 0 both	Item 27: uses a different team-membership join path and denies a valid team-member row.
Other	15	8 FD, 5 FP, 2 both	Item 5: swaps the meanings of `status` and `report_status`.
Total	190	76 FD, 31 FP, 75 both; 8 N/A	One primary category per failed item.

Among the 182 executable failed policies, 76 are false-deny only, 31 are false-permit only, and 75 exhibit both directions. In total, 151/182 contain at least one false denial and 106/182 contain at least one false permit. Wrong literals are the largest individual category; 32/55 differ from the reference only in letter case, often on text columns whose allowed value vocabulary is absent from the schema context. Explicit value vocabularies would avoid requiring the generator to infer annotation-specific casing. The zero count for `USING`/`WITH CHECK` placement indicates that operation-channel routing is not a remaining error source in this run.

Table 9. Worked false-permit example (Item 123).

Field	Case FP: Item 123
Requirement	Environmental health and safety officers can perform all row-level operations on safety-event records during weekday business hours (09:00–18:00).
Schema slice	`operations.security_events(event_id, record_type, ...)`; runtime context `app.employee_id`, `app.job_role`; trusted `app_now()`.
Reference policy	`FOR ALL USING/WITH CHECK`: role is `'ehs_officer'` **and** `record_type = 'Safety'` **and** time is 09:00–18:00 on weekdays.
CARGO prediction	Preserves the role and time predicates in both channels but omits `record_type = 'Safety'`.
Observed behavior	2/9 tests fail. Non-safety rows satisfy the broadened predicate, producing a pure false-permit failure.
Improvement target	Constraint-completeness checking should align every requirement-grounded object anchor with at least one generated predicate.

Table 10. Worked false-deny example (Item 445).

Field	Case FD: Item 445
Requirement	Sales representatives may create orders only for their tenant.
Schema slice	`sales.orders(order_id, tenant_id, sales_rep_id, ...)`; runtime context `app.employee_id`, `app.job_role`, `app.tenant_id`.
Reference policy	`FOR INSERT WITH CHECK`: role is `'sales_rep'` and the new row's `tenant_id` equals `app.tenant_id`.
CARGO prediction	Adds `sales_rep_id = app.employee_id` to the reference conditions.
Observed behavior	1/6 tests fails. A tenant-valid order assigned to another representative is rejected, producing a pure false-deny failure.
Improvement target	An anchor should not become a row-equality conjunct unless the requirement semantically links the actor to that column.

The taxonomy is produced by a deterministic rule-based classifier with fixed precedence, followed by manual inspection of every representative case shown above. It is exhaustive for this run but is not presented as an independently annotated human corpus. Some items contain multiple defects; the table records only the first category under the fixed precedence.

References

[1] V. Hu et al., "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST Special Publication 800-162, 2014 (updated 2019). https://doi.org/10.6028/NIST.SP.800-162

[2] N. R. Adam, V. Atluri, E. Bertino, and E. Ferrari, "A Content-Based Authorization Model for Digital Libraries," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, no. 2, pp. 296–315, 2002. https://doi.org/10.1109/69.991718

[3] PostgreSQL Global Development Group, "System Administration Functions," PostgreSQL Documentation. https://www.postgresql.org/docs/current/functions-admin.html

[4] PostgreSQL Global Development Group, "Customized Options," PostgreSQL Documentation. https://www.postgresql.org/docs/current/runtime-config-custom.html

[5] Supabase, "Supabase Studio: AI Assistant and User Impersonation," Dec. 2023. https://supabase.com/blog/studio-introducing-assistant

[6] Supabase, "Supabase AI Assistant v2," Dec. 2024. https://supabase.com/blog/supabase-ai-assistant-v2

[7] Supabase, "Feature Preview: RLS Tester," Apr. 2026. https://supabase.com/changelog/45233-feature-preview-rls-tester

[8] OHDSI, “OMOP Common Data Model v5.4 PostgreSQL DDL,” release v5.4.0. https://github.com/OHDSI/CommonDataModel/blob/v5.4.0/inst/ddl/5.4/postgresql/OMOPCDM_postgresql_5.4_ddl.sql

[9] PostgreSQL Global Development Group, “Row Security Policies.” https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[10] Y. Zhan, L. Cui, H. Weng, G. Wang, Y. Tian, B. Liu, Y. Yang, X. Yin, J. Xie, and Y. Sun, "Towards Database-Free Text-to-SQL Evaluation: A Graph-Based Metric for Functional Correctness," in *Proceedings of COLING*, 2025, pp. 4586–4610.

[11] T. Yu et al., "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proceedings of EMNLP*, 2018, pp. 3911–3921.

[12] R. Zhong, T. Yu, and D. Klein, "Semantic Evaluation for Text-to-SQL with Distilled Test Suites," in *Proceedings of EMNLP*, 2020, pp. 396–411.